```
Installing mustermann 1.0.0
Fetching rack 2.0.3
Installing rack 2.0.3
Using rspec-support 3.6.0
Fetching tilt 2.0.7
Installing tilt 2.0.7
Fetching rack-protection 2.0.0
Installing rack-protection 2.0.0
Fetching rack-test 0.7.0
Installing rack-test 0.7.0
Using rspec-core 3.6.0
Using rspec-expectations 3.6.0
Using rspec-mocks 3.6.0
Fetching sinatra 2.0.0
Installing sinatra 2.0.0
Using rspec 3.6.0
Bundle complete! 4 Gemfile dependencies, 14 gems now installed.
Use `bundle info [gemname]` to see where a bundled gem is installed.
```

Now, set up the project to use RSpec. For now, we'll always run rspec using bundle exec, to make sure we're using the exact library versions we're expecting. In *Bundler*, on page 293, we'll talk about another, quicker way to run our specs.

```
$ bundle exec rspec --init
  create   .rspec
  create   spec/spec_helper.rb
```

This command will generate two files:

- .rspec, which contains default command-line flags
- spec/spec_helper.rb, which contains configuration options

The default flags in .rspec will cause RSpec to load spec_helper.rb for us before loading and running our spec files.

You'll need to add one line to the top of spec/spec_helper.rb:

04-acceptance-specs/01/expense_tracker/spec/spec_helper.rb

```
ENV['RACK_ENV'] = 'test'
```

Now that all the pieces are in place, we can write our first example.

Use the Right Rack Environment



Setting the RACK_ENV environment variable to test switches on test-friendly behavior in your web framework. Sinatra normally swallows exceptions and renders a "500 Internal Server Error" response. With this variable set, Sinatra will instead allow errors to bubble up to your test framework.